Visualizador de un UR3e por TCP/IP

Memoria · Redes de Comunicación Industriales · Máster en Automatización, Electrónica y Control Industrial · Universidad de Deusto
Manuel Narváez · noviembre de 2026 · Código: github.com/ManuelN12/rci-visualizador-ur3e

1. Introducción

Se ha desarrollado en Python un visualizador que se conecta por TCP/IP a un UR3e (real o simulado), decodifica byte a byte sus mensajes binarios y reproduce su configuración articular en PyBullet. Se opta al **sobresaliente**; el trabajo es individual.

Nivel	Requisito	Estado
Aprobado (5)	Interfaces primaria (30001) y de tiempo real (30003), búfer y delimitación por longitud, robustez ante caídas	Validado con URSim (3/11)
Notable (7)	Varios robots en la misma ventana, configurados por fichero, un hilo por robot	Validado con dos URSim, el mío y el de un compañero (11/11)
Sobresaliente (10)	Envío de URScript y mini-aplicación «Enseñar y reproducir» con supervisión	Validado con URSim y con el UR3e real del laboratorio (11/11)

Entorno. URSim 5.25.2 (PolyScope 5.25, modelo UR3) en VirtualBox 7.2.22. Cliente en un portátil Windows con Python 3.12 en un entorno conda (Miniforge), `pybullet` 3.25 de conda-forge y el paquete `ur3e_rg2` proporcionado. Para la comunicación solo se usan los módulos estándar `socket`, `struct` y `threading`. Para el PC del laboratorio, que no tiene Python, el programa se empaqueta en un ejecutable con PyInstaller (`construir_exe.bat`).

2. Montaje de red

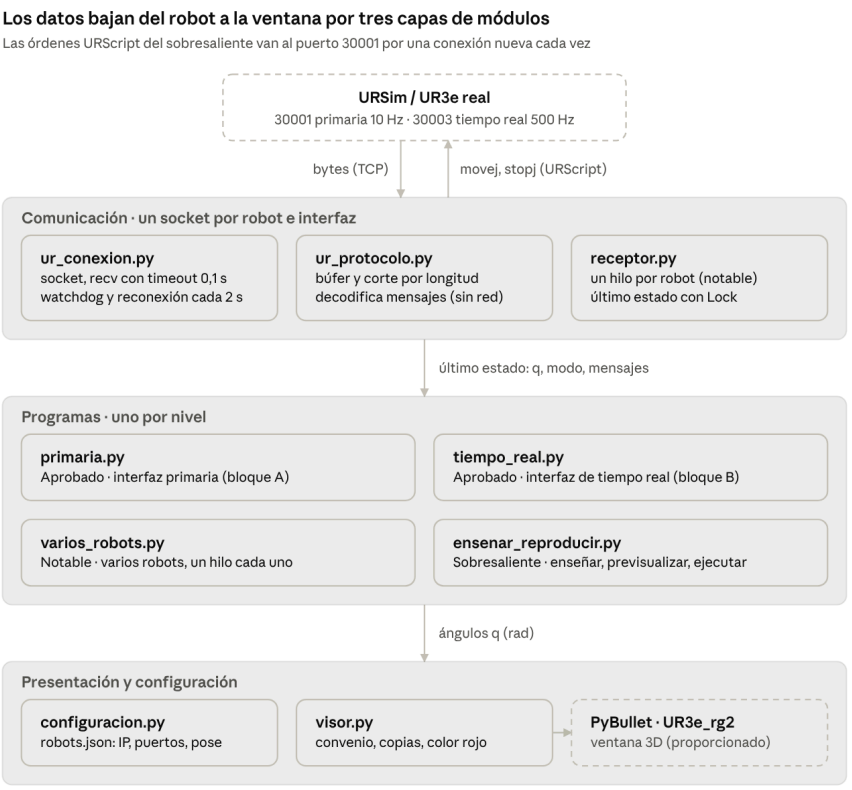
URSim se ejecuta en una VM con el adaptador en **modo puente**: es un equipo más de la red, con su propia MAC e IP, y el programa lo alcanza igual que a un UR3e real (la VM venía en NAT y hubo que cambiarla).

Lugar	PC cliente	VM (URSim)	Red
Portátil, Wi-Fi (3/11)	10.164.49.233	10.164.57.193	10.164.48.0/20: el tercer octeto difiere, pero es la misma subred
Aula, cable (11/11)	10.172.18.61	10.172.18.62	10.172.18.0/24, junto a la VM del compañero (10.172.18.59)

Antes de ejecutar se comprueba la red por capas: `ping` (4/4, < 1 ms, TTL = 64, un equipo Linux) y `Test-NetConnection -Port 30001` y `-Port 30003` (`TcpTestSucceeded : True`). Para el notable hubo que pasar del Wi-Fi al cable: desde el Wi-Fi la VM del compañero daba *timeout*, porque la universidad filtra el tráfico entre ambas redes. Para el sobresaliente, en PolyScope: *Settings → System → Remote Control* habilitado y el robot en **Remote**. La IP y los puertos de cada robot están en `config/*.json`: cambiar de red o pasar al robot real solo cambia ese fichero, no el código.

3. Estructura del código

El código se reparte en tres capas: la comunicación no sabe nada de PyBullet, la decodificación no sabe nada de la red, y cada nivel de la práctica es un programa propio que reutiliza las capas inferiores.



`primaria.py` y `tiempo_real.py` usan `ur_conexion.py` directamente en el hilo principal; `varios_robots.py` y `ensenar_reproducir.py` crean un `ReceptorRobot` (un hilo) por robot. Solo el hilo principal llama a PyBullet, a través de `visor.py`, que aplica el convenio de ángulos y pasa copias de las listas porque `move_robot()` las modifica. Para probar sin URSim, `herramientas/robot_falso.py` imita las dos interfaces, puede enviar los bytes en trozos aleatorios y provocar errores, paradas y el modo Local.

4. Estrategia de lectura

TCP entrega un flujo de bytes, no mensajes, así que las dos interfaces se leen con el mismo búfer delimitado por longitud (`ur_protocolo.BufferTramas`):

1. `recv()` y añadir lo recibido al final de un `bytearray`. Si devuelve 0 bytes, el robot ha cerrado la conexión.
2. Mientras haya al menos una cabecera (5 B en la primaria, 4 B en tiempo real), leer la longitud `L` (`>i`, big-endian) sin consumirla.
3. Si `L` es imposible (menor que la cabecera o mayor que 64 kB), el búfer está desalineado: cerrar y reconectar.
4. Si hay menos de `L` bytes, esperar al siguiente `recv()`; si hay `L` o más, cortar el mensaje, quitarlo del búfer y volver al paso 2.

Interfaz primaria (30001, 10 Hz). Cabecera `>iB` (longitud y tipo). Del mensaje de estado (tipo 16) se recorren los subpaquetes saltándolos por su longitud. Del subpaquete 1 (*Joint data*, 251 B = 5 + 6 × 41) se lee el primer `double` de cada bloque `>ddddffffB`; del subpaquete 0 (*Robot mode*), el modo, las paradas y si hay un programa en marcha. Los mensajes de tipo 20 (versión, error, aviso, excepción) se muestran; cualquier otro tipo se salta.

Interfaz de tiempo real (30003, 500 Hz). Paquetes de estructura fija de 1220 B: `q_actual` son 6 `double` desde el byte 252. Se consumen todos los paquetes recibidos, se guarda solo el último y la ventana se redibuja como mucho 60 veces por segundo, así que el robot simulado no acumula retraso.

Robustez (`ur_conexion.ConexionUR`). `recv()` tiene un timeout de 0,1 s para no bloquear la ventana. La conexión se cierra (vaciando el búfer) y se reintenta cada 2 s si el robot la cierra, si el búfer se des-

alinea o si el *watchdog* detecta silencio: más de 2 s sin datos en la primaria o 0,5 s en tiempo real (cable cortado o robot colgado, casos en los que no llega ningún aviso de cierre).

Varios robots (notable). Cada robot tiene un hilo con su socket y su búfer, que solo escribe el último estado. El hilo principal lee una copia a 30 Hz y es el único que llama a PyBullet. El estado compartido se protege con `threading.Lock`, y dentro de la sección crítica solo se asignan o copian valores. Si un robot pasa más de 1 s sin datos, su modelo se queda quieto y se pinta de rojo.

Órdenes (sobresaliente). Cada `movej(q, a=0.5, v=0.3)` se envía por una **conexión nueva** al puerto 30001 (`ConexionUR.enviar_orden`). El motivo se descubrió en el robot real (apartado 5): el controlador fija al conectar si una conexión puede dar órdenes.

5. Pruebas y resultados

Aprobado y robustez (URSim, 3/11).

Prueba	Resultado
A2 · mensajes	Tipo 16 de 722 B (a veces 743 B) a 10 Hz; tipo 5 de 9 B, no documentado, se salta por su longitud
A3 · subpaquetes	12 subpaquetes que suman 717 = 722 − 5 en todos los mensajes; *Joint data* = 251 B
A4–A5 · ángulos y visor	Coinciden con la pestaña *Move* de PolyScope y PyBullet sigue a URSim sin corrección de ángulos
B1–B3 · tiempo real	Todos los paquetes miden 1220 B; ángulos iguales a los de la primaria (byte 252)
B2 · frecuencia	270–320 paquetes/s, no 500: el reloj de URSim avanza solo 0,53–0,63 s por segundo real; `recv llenos` = 0
B4 · retraso	Más de un minuto moviendo rápido: el modelo se detiene a la vez que URSim
Cerrar PolyScope	Los datos siguen llegando, y más rápido (≈ 430 paquetes/s): el controlador es un proceso aparte
Reinicio brusco de la VM	No llega FIN; el watchdog cierra a los 0,6 s y reintenta cada 2 s sin bloquear la ventana
Reinicio del controlador	`recv` devuelve 0 bytes y el programa se reconecta solo en 3 s

En A4, PolyScope mostraba Wrist 3 = 358,22° con la marca −1R y por la red llegaba −1,78°: es el mismo ángulo. PolyScope muestra 0–360° más el número de vueltas, y la interfaz envía el valor acumulado.

Notable (11/11). `varios_robots.py` mostró en la misma ventana mi URSim y el de un compañero (colocado a 0,8 m y girado 180°, según `config/dos_ursim.json`). Cada modelo siguió a su robot. Al cortar la red del compañero, su modelo se quedó quieto y en rojo a 1 s sin datos y la conexión se cerró a los 2 s; mientras tanto, mi robot siguió a ≈ 9 msg/s. Al volver la red, se reconectó y recuperó el color. Un brazo apagado (`POWER_OFF`) no se pone rojo, porque el controlador sigue publicando su estado.

Sobresaliente con URSim (11/11). Se enseñaron 4 puntos (3 desde PolyScope y 1 con los deslizadores de PyBullet), se previsualizaron con el robot fantasma, se guardaron, se recargaron y se ejecutaron: los 4 se confirmaron con error 0,000°. Cada `movej` genera los avisos `PROGRAM_XXX_STARTED` y `PROGRAM_XXX_STOPPED` (tipo 20, subtipo 7). El tramo más largo (2,51 rad a 0,3 rad/s, 8,4 s teóricos) tardó 14 s, coherente con URSim al ≈ 60 % del tiempo real. **Error provocado:** al apagar el robot a mitad de un tramo, el controlador pausó el programa con el robot a 141° del objetivo y la secuencia se detuvo; `seguir` se rechazó mientras el robot estaba en `POWER_OFF`, y tras ON → START reenvió el punto y terminó con normalidad.

UR3e real (laboratorio, 11/11). El mismo ejecutable, cambiando solo la IP en `config/ur3e_real.json`, siguió al robot real. Al ejecutar apareció una diferencia con URSim: el robot rechazaba los `movej` con el error **C210A0** (*socket is read-only when the robot is in local control*) aunque ya estaba en Remote. El controlador fija el permiso de cada conexión al abrirla, y el programa se había conectado con el robot en Local para enseñar los puntos. Se corrigió enviando cada orden por

una conexión nueva; así se enseñaron los puntos en Local y se ejecutó la secuencia en Remote sin reiniciar el programa. La defensa (13/11) se hizo con URSim porque el laboratorio estaba ocupado.

6. Respuestas a las preguntas de comprensión

1. Bytes de cada `recv()`. No siempre coinciden con un mensaje. En la primaria, 946 llamadas en 55 s devolvieron 9, 722 o 743 B, pero las cinco primeras devolvieron 136, 181, 881, 1460 y 17 B. En tiempo real, la primera devolvió 3660 B (3 paquetes juntos). En régimen normal cada `recv` trae un mensaje porque llegan espaciados, y un programa ingenuo *parecería* funcionar; al conectar llegan trozos que cortan mensajes (1460 B es el MSS: 1500 de MTU − 40 de cabeceras IP y TCP) o varios juntos. Consecuencia: hay que acumular en un búfer y cortar por la longitud. Con el robot falso enviando trozos aleatorios, el búfer reconstruye todos los mensajes.

2. Por qué empiezan con su longitud. TCP no conserva los límites de los mensajes: la longitud es lo único que dice dónde acaba uno y empieza el siguiente. Además hace el protocolo extensible: un cliente puede saltar lo que no entiende sin desalinearse, como hace nuestro programa con el mensaje de tipo 5 y con el subpaquete extra de 21 B de los mensajes de 743 B.

3. Sin indicar big-endian. Salen valores absurdos. −90° = −π/2 rad viaja como `bf f9 21 fb 54 44 2d 18`: con `>d` se lee −1,5708 rad; con `<d` (orden de un PC x86) sale $3{,}2 \times 10^{-192}$. El signo y el exponente se toman de bytes de la mantisa. Por eso todos los formatos de `struct` empiezan por `>`.

4. Primaria frente a tiempo real. La primaria va a 10 Hz con formato variable (mensajes y subpaquetes), trae el estado completo y los errores, y genera ≈ 7 kB/s. La de tiempo real va a 500 Hz con formato fijo de 1220 B, solo trae datos numéricos y genera ≈ 600 kB/s. Para un **panel de supervisión**, la primaria: basta para el ojo, consume poco y trae los errores. Para **registrar la trayectoria de un movimiento rápido**, la de tiempo real: una muestra cada 2 ms frente a cada 100 ms.

5. NAT frente a puente. En NAT la VM está detrás de un router virtual con una IP privada (10.0.2.15) que el resto de la red no ve; las conexiones entrantes no llegan sin reenvío de puertos, y URSim es un servidor que las espera. En puente, la tarjeta virtual sale a la red física con su propia MAC y obtiene su propia IP. Ven esa MAC los equipos de su dominio de difusión: el switch, que la aprende en el puerto del PC, y los equipos de la subred que la resuelven por ARP. Un router no la propaga fuera de la subred.

6. Red cortada 5 s. Se desmarcó el cable virtual de la VM con `tiempo_real.py` en marcha. No llegó ningún aviso de cierre: el watchdog detectó 0,6 s sin datos, cerró la conexión y el modelo se quedó quieto. El programa reintentó cada 2 s (`timed out`, porque el SYN no recibe respuesta) sin bloquear la ventana, y se reconectó solo menos de 2 s después de volver la red. TCP, durante el corte, guarda los datos no confirmados en el búfer de envío del robot y los retransmite con esperas crecientes. Si la conexión siguiera abierta, al volver la red llegarían de golpe datos viejos; al cerrarla y abrir otra, se descartan y se recibe directamente el estado actual.

7. (N) Retraso. Es la suma del periodo de muestreo (hasta 100 ms en la primaria, 2 ms en tiempo real), la red local (< 1 ms) y el refresco de la ventana (hasta 33 ms a 30 Hz, 17 ms a 60 Hz): hasta ≈ 135 ms con la primaria y ≈ 20 ms con la de tiempo real. Depende sobre todo de la frecuencia de la interfaz y de que el cliente consuma todo lo que llega: con `recv llenos` = 0 no se acumulan datos atrasados. Comparar los relojes del PC y de URSim no sirvió, porque el de URSim avanza a 0,6 s por segundo real.

8. (N) Sin Lock. El hilo principal podría leer un estado a medio escribir: ángulos nuevos con la marca de tiempo vieja, y tomar por «con datos» un robot que acaba de perderlos. Recorrer la lista de mensajes mientras el receptor añade otro lanzaría `RuntimeError: deque mutated during iteration`. El GIL hace atómica una asignación suelta, no un grupo de ellas. Con el Lock, el receptor actualiza todos los campos juntos y el principal copia todo el estado de una vez.

9. (S) Enviar un `movej` no basta. `sendall` solo garantiza que los bytes salieron del PC. El robot puede no ejecutar la orden (modo Local, parada, error, punto inalcanzable), y aunque la ejecute, el movimiento tarda segundos. Se confirma con el estado que publica el propio robot: antes de enviar, datos

recientes, modo `RUNNING`, sin paradas ni programa en marcha; después, `isProgramRunning` pasa a verdadero y luego a falso, y `q_actual` queda a menos de 0,01 rad del objetivo. Un error (tipo 20, subtipo 6 de nivel ≥ 3, o subtipo 10), una parada o el C210 detienen la secuencia. Se comprobó apagando el robot a mitad de un movimiento.

10. (S) Cualquier equipo puede enviar programas al 30001. El puerto no tiene autenticación: un PC infectado o un tercero en la red de planta podría mover, parar o reprogramar el robot, con riesgo para las personas, el producto y la producción. **En el robot:** mantenerlo en Local salvo cuando haga falta el control remoto, con contraseña en PolyScope, desactivar en *Settings → Security* los servicios que no se usen y restringir el acceso entrante a las IP autorizadas. **En la red:** aislar los robots en una red o VLAN de planta, con un cortafuegos que solo deje llegar a sus puertos desde los equipos de supervisión (segmentación por zonas y conductos, IEC 62443). La universidad ya filtra el tráfico entre el Wi-Fi y la red del aula.

7. Conclusiones

El visualizador funciona con URSim y con el UR3e real en las dos interfaces, y sobrevive a caídas abruptas y ordenadas del robot sin colgarse. La lección central es que **TCP entrega bytes, no mensajes**: aunque en régimen normal cada `recv` parezca traer un mensaje, al conectar aparecieron trozos de 1460 B y bloques de 3 paquetes, y solo el búfer con delimitación por longitud los interpreta bien.

- La frecuencia baja de tiempo real se debía a los recursos de la VM, no al cliente: `recv llenos` = 0, y al cerrar PolyScope subió de ≈ 300 a ≈ 430 paquetes/s.
- Separar recepción (un hilo por robot) y dibujo (hilo principal), con un Lock y copias, hace que un robot caído no afecte a los demás.
- Para mandar órdenes no basta con enviarlas: hay que cerrar el lazo con el estado que publica el robot.
- El robot real aportó la diferencia más instructiva con URSim: el permiso de escritura de una conexión se decide al abrirla, así que las órdenes se envían por una conexión nueva cada vez.